Name: Varad Gajanan Shelar

Reg no.: 24BCE0590

OS Lab Assesment 5

Process Synchronization using Semaphores

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define ITEMS_PER_PRODUCER 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    int producer_id = *(int *)arg;

    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++) {
        int item = producer_id * 100 + i;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer %d produced %d at position %d\n",
               producer_id, item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
```

```c
    return NULL;
}

void *consumer(void *arg) {
    int consumer_id = *(int *)arg;

    for (int i = 1; i <= ITEMS_PER_PRODUCER; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer %d consumed %d from position %d\n",
               consumer_id, item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main() {
    pthread_t producer_thread;
    pthread_t consumer_thread;

    int producer_id = 1;
    int consumer_id = 1;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(
        &producer_thread,
        NULL,
        producer,
        &producer_id
    );
```

```
    pthread_create(
        &consumer_thread,
        NULL,
        consumer,
        &consumer_id
    );

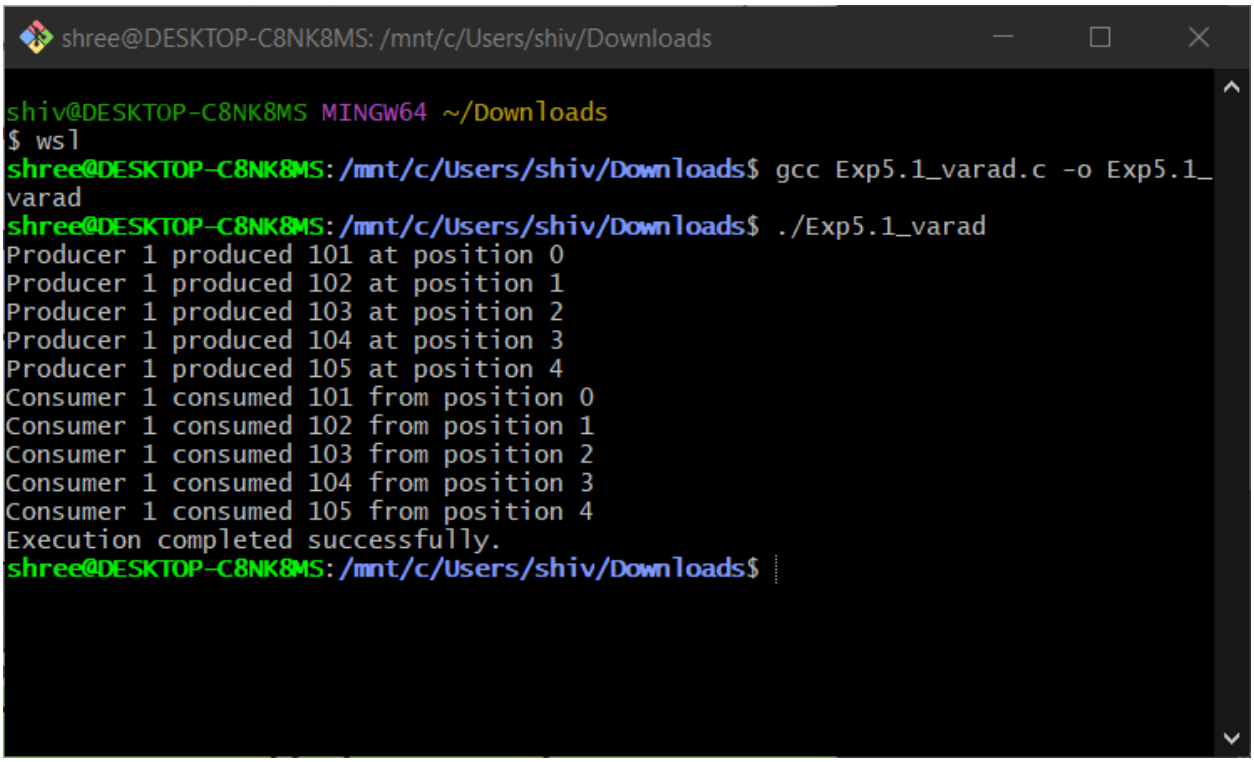
    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("Execution completed successfully.\n");

    return 0;
}
```

Output:



Data Parallelism using Multithreading

Code:

```c
/*
Experiment: Data Parallelism Using Multithreading
Application: Parallel Electricity-Bill Calculation

The customer data is divided into independent portions.
Each thread calculates the electricity bills for its assigned customers.
*/

#include <stdio.h>
#include <pthread.h>

#define MAX_CUSTOMERS 100
#define MAX_THREADS 10

typedef struct {
    int customerId;
    float units;
    float bill;
} Customer;

typedef struct {
    int threadId;
    int startIndex;
    int endIndex;
} ThreadData;

Customer customers[MAX_CUSTOMERS];
int numberOfCustomers;

float calculateBill(float units) {
    float bill;

    if (units <= 100) {
        bill = units * 1.50;
    }
    else if (units <= 200) {
        bill = (100 * 1.50) +
               ((units - 100) * 2.50);
    }
```

```
    else if (units <= 300) {
        bill = (100 * 1.50) +
               (100 * 2.50) +
               ((units - 200) * 4.00);
    }
    else {
        bill = (100 * 1.50) +
               (100 * 2.50) +
               (100 * 4.00) +
               ((units - 300) * 6.00);
    }

    return bill;
}

void *processCustomers(void *argument) {
    ThreadData *data = (ThreadData *)argument;

    printf("Thread %d is processing customers %d to %d\n",
           data->threadId,
           data->startIndex + 1,
           data->endIndex);

    for (int i = data->startIndex; i < data->endIndex; i++) {
        customers[i].bill = calculateBill(customers[i].units);
    }

    return NULL;
}

int main() {
    int numberOfThreads;
    pthread_t threads[MAX_THREADS];
    ThreadData threadData[MAX_THREADS];

    printf("Enter the number of customers: ");
    scanf("%d", &numberOfCustomers);

    if (numberOfCustomers <= 0 ||
        numberOfCustomers > MAX_CUSTOMERS) {
        printf("Invalid number of customers.\n");
        return 1;
    }
```

```c
for (int i = 0; i < numberOfCustomers; i++) {
    customers[i].customerId = i + 1;

    printf("Enter electricity units for customer %d: ", i + 1);
    scanf("%f", &customers[i].units);

    if (customers[i].units < 0) {
        printf("Electricity units cannot be negative.\n");
        return 1;
    }
}

printf("Enter the number of threads: ");
scanf("%d", &numberOfThreads);

if (numberOfThreads <= 0 ||
    numberOfThreads > MAX_THREADS) {
    printf("Number of threads must be between 1 and %d.\n",
           MAX_THREADS);
    return 1;
}

if (numberOfThreads > numberOfCustomers) {
    numberOfThreads = numberOfCustomers;
    printf("Number of threads adjusted to %d.\n",
           numberOfThreads);
}

int customersPerThread =
    numberOfCustomers / numberOfThreads;

int remainingCustomers =
    numberOfCustomers % numberOfThreads;

int currentIndex = 0;

printf("\nDividing customer records among threads...\n");

for (int i = 0; i < numberOfThreads; i++) {
    int assignedCustomers = customersPerThread;
```

```
        if (i < remainingCustomers) {
            assignedCustomers++;
        }

        threadData[i].threadId = i + 1;
        threadData[i].startIndex = currentIndex;
        threadData[i].endIndex =
            currentIndex + assignedCustomers;

        currentIndex = threadData[i].endIndex;

        int status = pthread_create(
            &threads[i],
            NULL,
            processCustomers,
            &threadData[i]
        );

        if (status != 0) {
            printf("Error creating thread %d.\n", i + 1);
            return 1;
        }
    }

    for (int i = 0; i < numberOfThreads; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nElectricity Bill Report\n");
    printf("-----------------------------------------\n");
    printf("Customer ID\tUnits Used\tBill Amount\n");
    printf("-----------------------------------------\n");

    for (int i = 0; i < numberOfCustomers; i++) {
        printf("%d\t\t%.2f\t\tRs. %.2f\n",
               customers[i].customerId,
               customers[i].units,
               customers[i].bill);
    }

    printf("-----------------------------------------\n");
    printf("All customer bills calculated successfully.\n");
```

```
    return 0;
}
```

Output:

